

Objectives

Linked List

Linked List ADT

Applications of Linked List

Linked List – Java collections

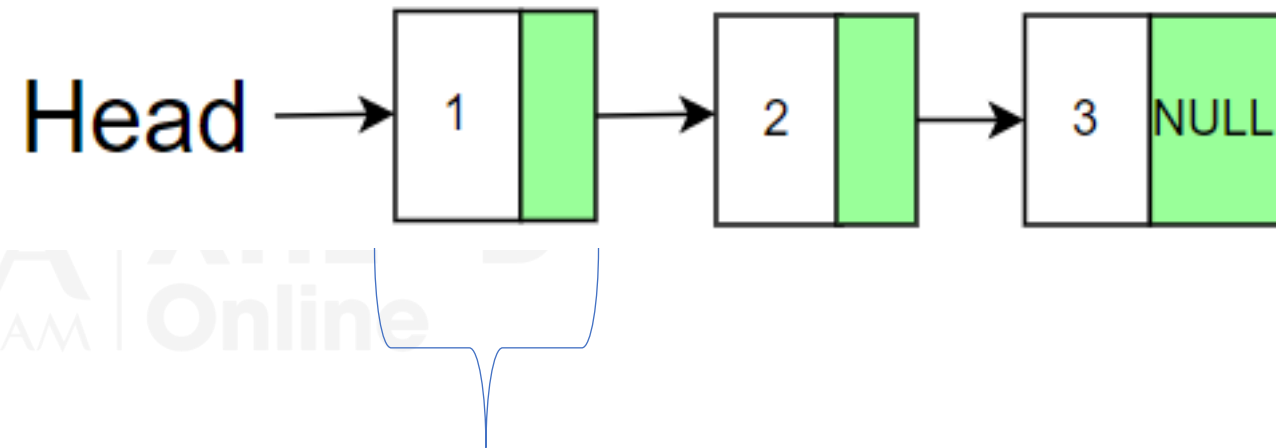
Linked List

- Linear Data Structure.
- Stores collection of items(list of items).
- Follows a linked representation.(successive items are not stored sequentially in adjacent locations as in arrays, items may be placed any where in the memory.)
- Each element of the list stores the address of next element for accessing list in correct order.

Linked List

Head - Stores the address of the first node.

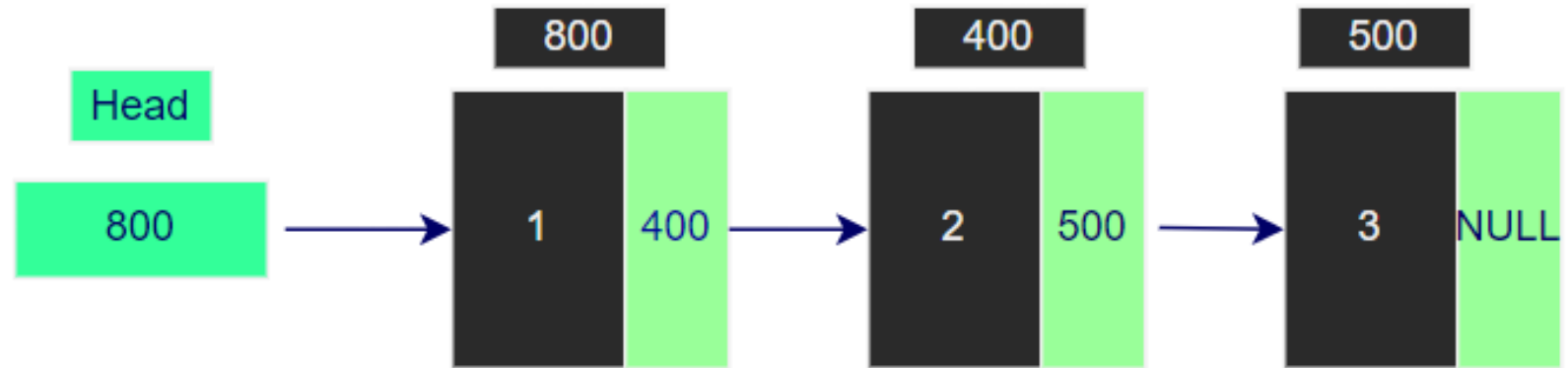
Node - Stores data and address to next node, which is used for accessing next data item. Address part of the last node is NULL. It marks the end of the list.



Node in a list, with a data part and next address part.

Linked List

Head variable stores the address of first node, 800. First node contains data value, 1 and address to next node ie, 400.



Linked List - Properties

- Accessed using a Head/Start pointer.
- Successive elements are connected by pointers.
- Next address field of last element points to NULL.
- Can grow or shrink dynamically.
- Elements are sequentially accessed.
- Random access is not possible.

Linked List -ADT

The following operations are supported by linked list:

- **Insert_to_list(item,pos)** //Insert item to a list in position **pos**.
If $pos==0$ then
 insert at the beginning of the list.
else If $pos==n$ then // n is number of nodes in the list,
 insert to end of the list.
else
 insert in position **pos**
- **item Del_from_list(pos)**
Delete an item at position pos and return the value deleted.
- **Traverse()** - Visit each node and do some action say, print the value.

Linked List -Applications

1. Implement data structures like Stack, Queue, Tree, Graph, Sparse Matrix
2. Memory Management in Operating System.
3. Process management in Operating System.



Linked List class Java

```
L = LinkedList()    #L is simply the variable name that holds the LinkedList  
                    #object you created using your custom LinkedList class.  
L.add(100)  
L.add(200)  
L.addFirst(1)      #add to beginning of the list  
  
L.print_list()
```

Linked List Methods

`print(L.getFirst())` # Output: 1

`print(L.getLast())` # Output: 200

`print(L.removeFirst())` # Output: 1

`print(L.removeLast())` # Output: 200

References

Data Structures & Problem Solving Using Java, Mark Allen Weiss
Fundamentals of Data Structures. Ellis Horowitz and Sartaj Sahni



Objectives

Linked List Implementation

Linked List Operations

insert

delete

traverse



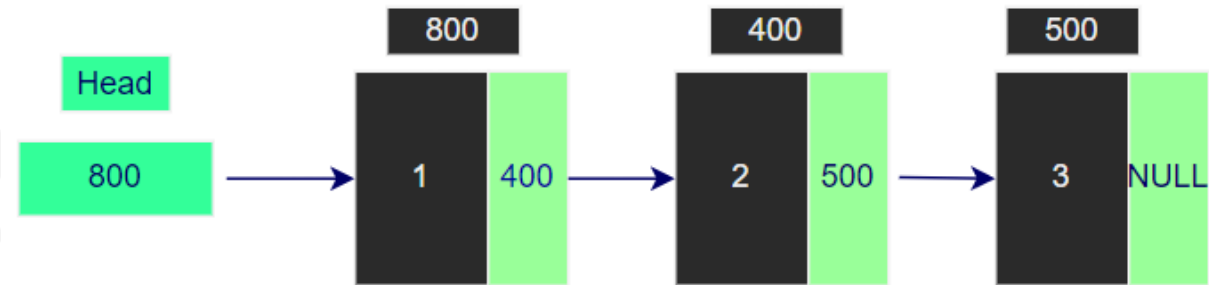
Linked List

- List consists of Nodes – design a

```
class Node:  
    def __init__(self, data):  
        self.data = data # Value of the node  
        self.next = None # Reference to the next node
```

- List is collection of Nodes –
design a class for a List

```
class LList:  
    def __init__(self):  
        self._head = None
```



Linked List – class Node

```
class Node:
    def __init__(self, data=0, next_node=None):
        self._data = data
        self._next = next_node

    def print(self):
        print(self._data)

    def set_next(self, p):
        self._next = p

    def get_next(self):
        return self._next

    def set_data(self, d):
        self._data = d

    def get_data(self):
        return self._data
```

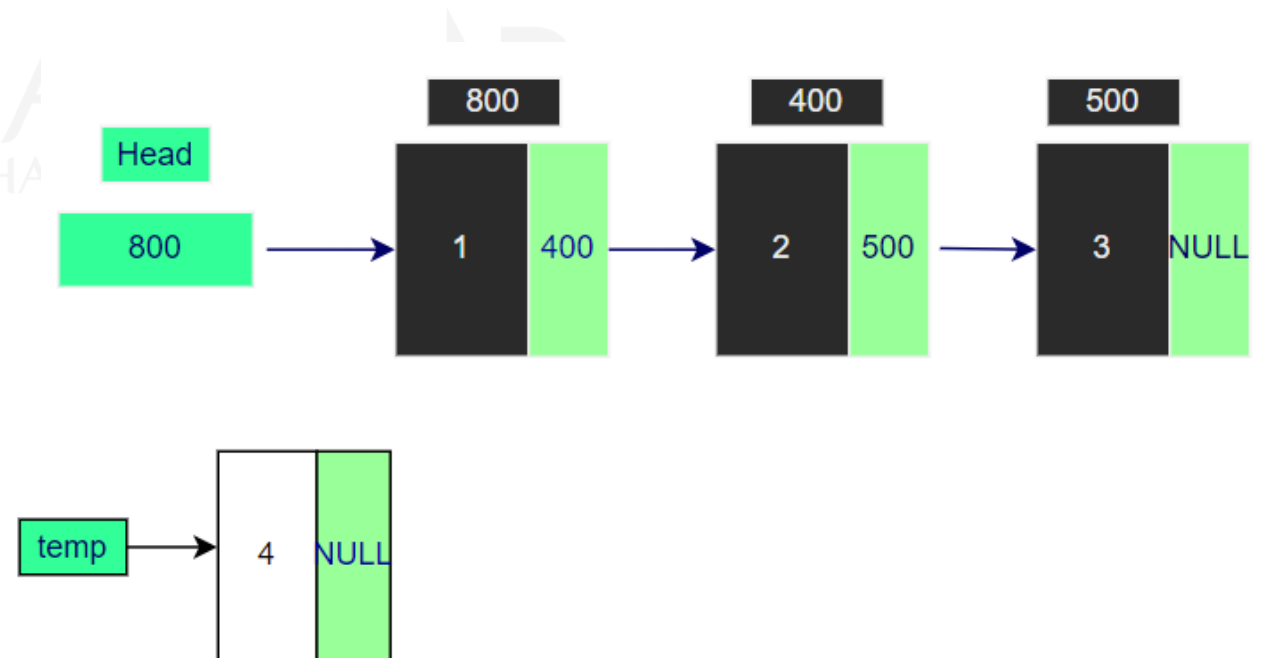
Linked List – class Linked List

```
class LList:  
    def __init__(self):  
        self._head = None  
  
    def Insert_to_list(self, item, pos):  
        # Placeholder for the method logic  
        pass
```

Place your
Webcam Video here
Size 38%

Linked List – insert

```
def insert_to_list(self, item, pos):  
    if pos == 0:  
        temp = Node(item, None)  
        temp.set_next(self._head)  
        self._head = temp
```

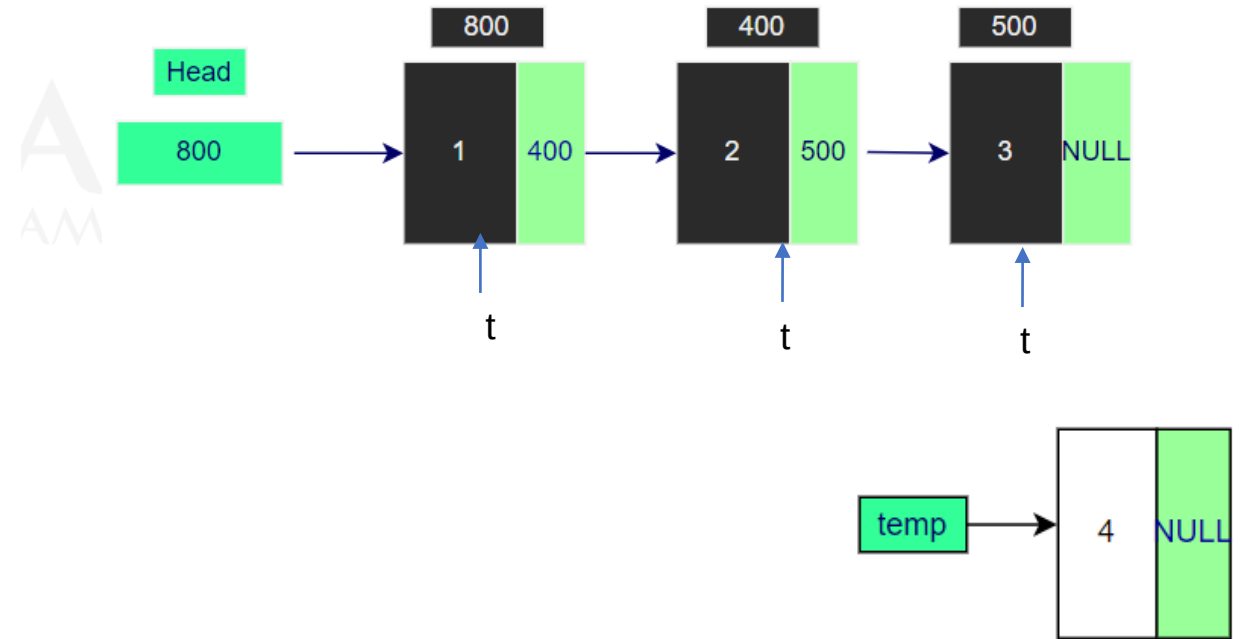


Linked List – insert

```
elif pos == self.size():
    t = self._head
    while t.get_next() is not None:
        t = t.get_next()
    temp = Node(item, None)
    t.set_next(temp)
```

You need to have a size() method to compute the current number of nodes in the list.
Here's a simple implementation:

```
def size(self):
    count = 0
    current = self._head
    while current:
        count += 1
        current = current.get_next()
    return count
```



Linked List – insert

else:

```
t = self._head
```

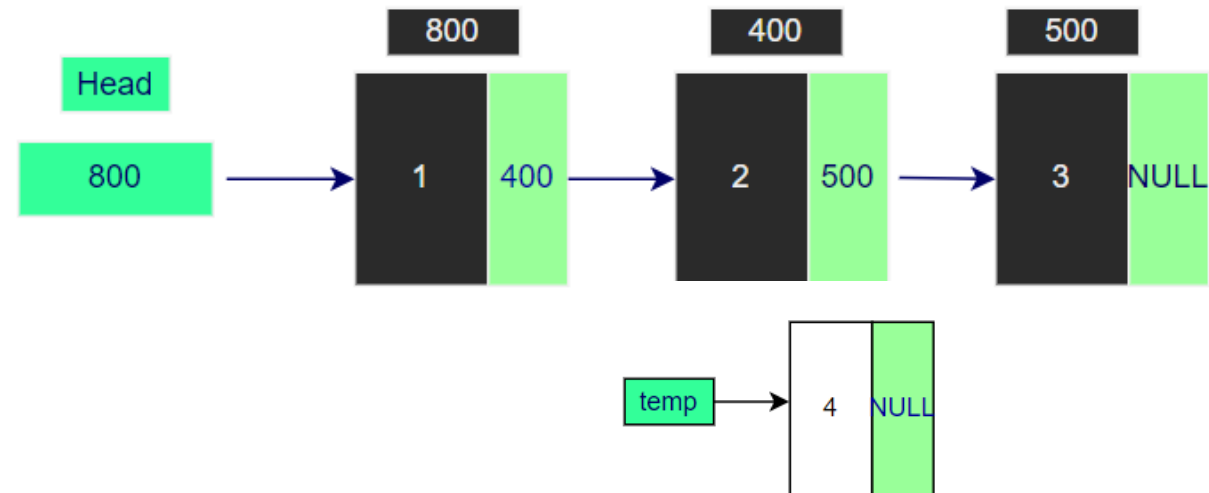
```
while t.get_next() is not None and pos > 1:
```

```
    t = t.get_next()
```

```
    pos -= 1
```

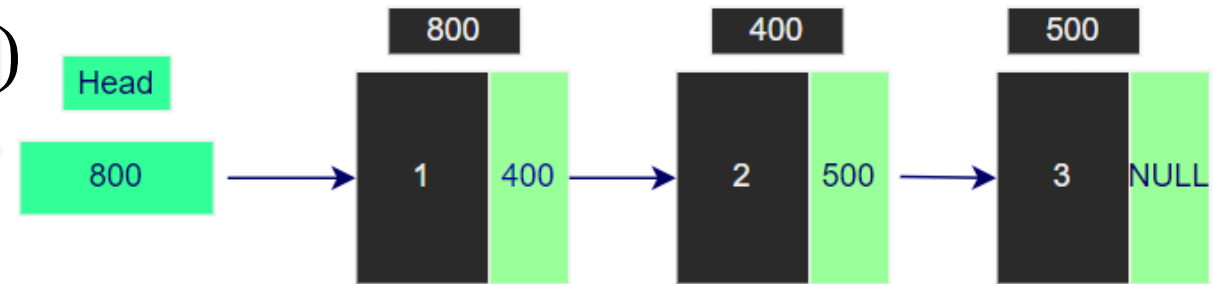
```
temp = Node(item, t.get_next())
```

```
t.set_next(temp)
```



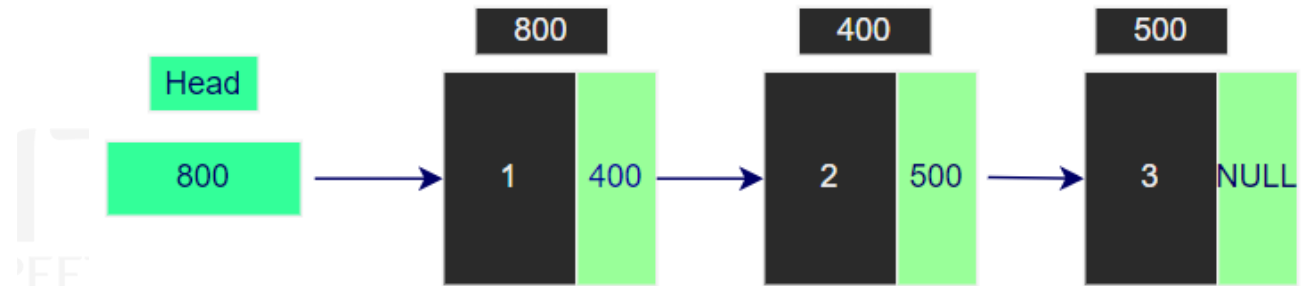
Linked List – delete

```
def del_from_list(self, p):  
    if self._head is None:  
        print("empty list")  
        return -1  
    elif p == 0:  
        val = self._head.get_data()  
        self._head = self._head.get_next()  
        return val
```



Linked List – delete

```
elif p == self.size() - 1:  
    t = self._head  
    prev = None  
    while t.get_next() is not None:  
        prev = t  
        t = t.get_next()  
    val = t.get_data()  
    if prev:  
        prev.set_next(None)  
    else:  
        self._head = None # In case list had only one node  
    return val
```



Linked List – delete

else:

temp = head

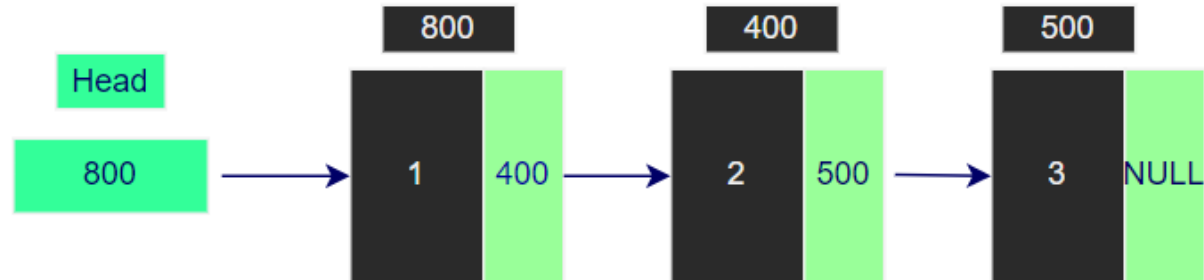
prev = head

while temp.getNext() is not None and $p > 1$:

prev = temp

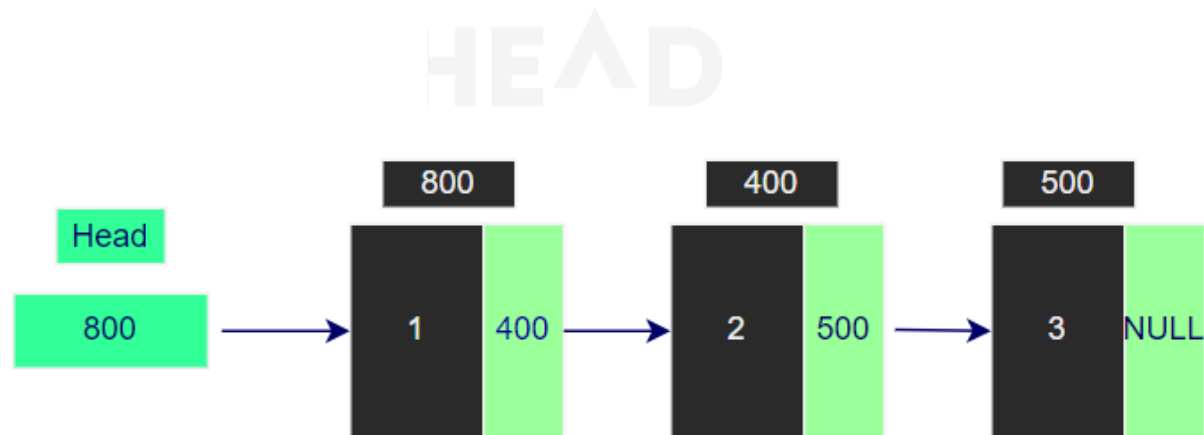
temp = temp.getNext()

$p -= 1$



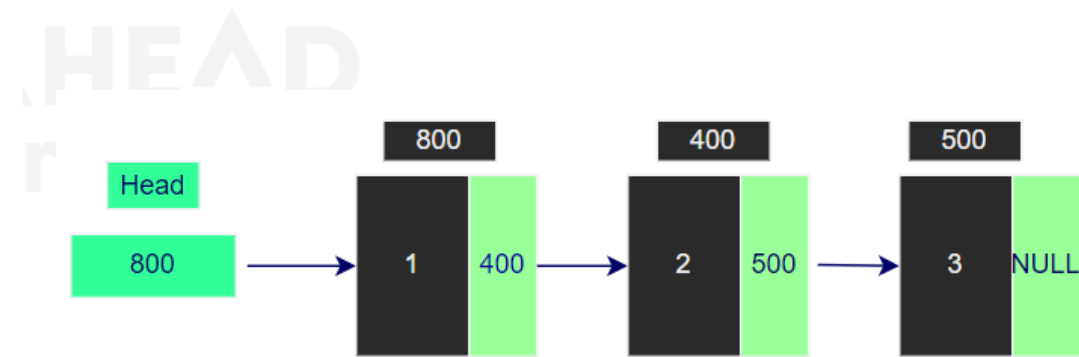
Linked List – delete

```
data = temp.getData()
if prev == temp:
    head = head.getNext()
else:
    prev.setNext(temp.getNext())
return data
```



Linked List – Traversal

```
def print_list():  
    temp = head  
    while temp is not None:  
        print(temp.getData())  
        temp = temp.getNext()
```



Summary

We have learnt how to implement a linked list and perform different operations on the linked list.



References

Data Structures & Problem Solving Using Java, Mark Allen Weiss
Fundamentals of Data Structures. Ellis Horowitz and Sartaj Sahni



Objectives

Loop Detection in Linked List



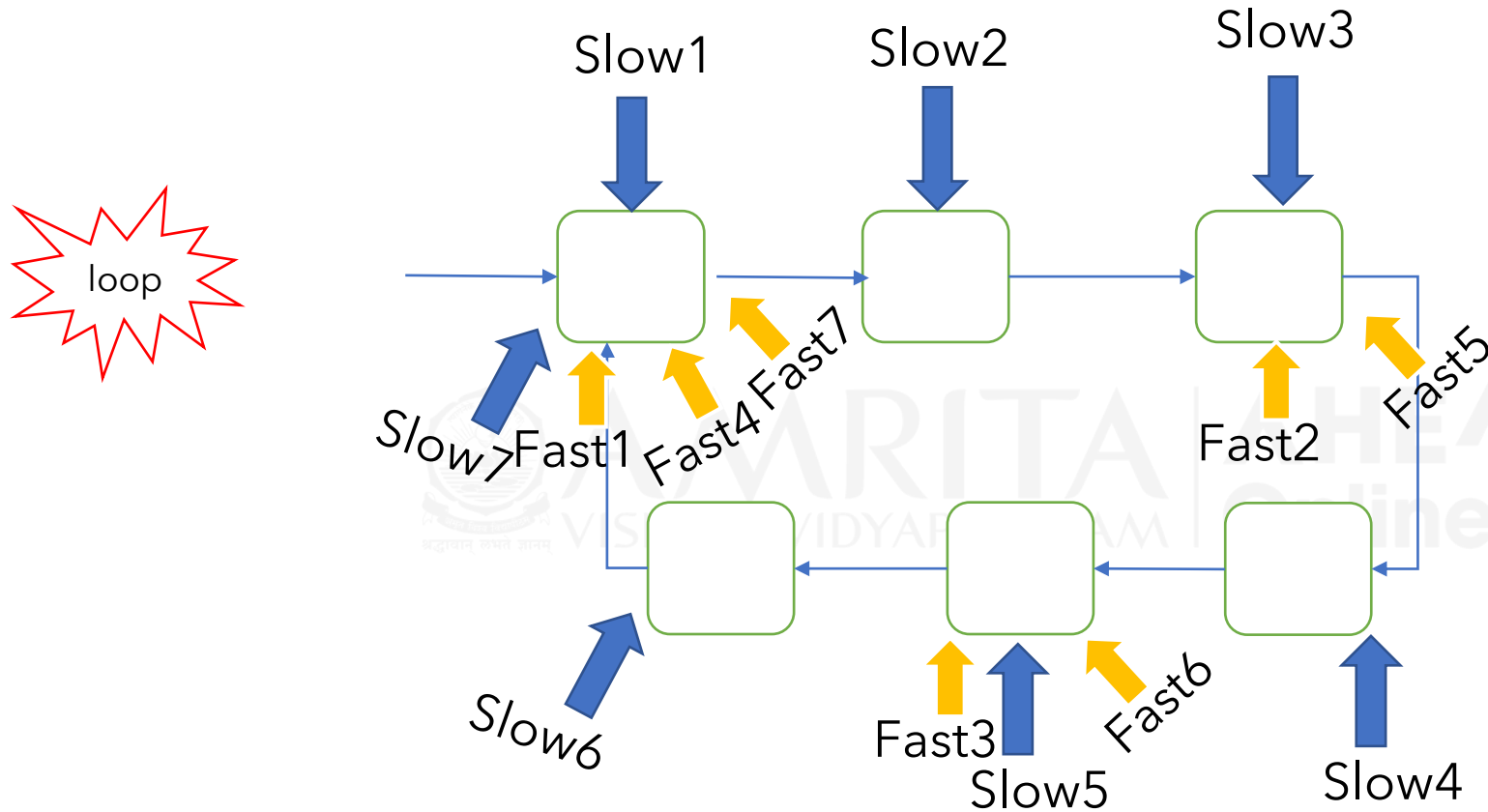
Loop Detection in Linked List

Floyd's Cycle-Finding Algorithm

1. Traverse the linked list using 2 references, Slow and Fast.
2. Fast reference moves by skipping one node in the list.
3. Slow reference moves without skipping, just moves to next node.
4. If the two references meet at a node then they are moving in a cycle and a loop exists in the linked list.



Loop Detection in Linked List – How it works?



Loop Detection in Linked List – Implementation

```
def loop_check(lst):  
    slowref = lst.getHead()  
    fastref = slowref  
  
    while slowref is not None and fastref is not None and fastref.getNext() is not None:  
        slowref = slowref.getNext()  
        fastref = fastref.getNext().getNext()  
        if slowref == fastref:  
            return True  
    return False
```

Summary

Determine a loop in a list.



Reference

Data Structures and Algorithms made easy - Narasimha Karumanchi

